Engineer to Excel

SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Accredited by NAAC A Grade

SIMATS ENGINEERING

ASSESSMENT TOOL 2 - Technical Presentation

COURSE NAME: Data Structures **COURSE CODE: CSA03**

Register Number	**1921260133**
Name	**Abinayasri K**

Assessment Tool Weightage: 30%

COURSE OUTCOME COVERED

CO1: *Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL3, BL4)*

Questions:4 **EACH QUESTIONS CARRIES:** 25 MARKS **Total Marks:**100 **Duration:** ONE HOUR

S. No.	List of Questions
1	A student management system needs to store the marks of 1000 students efficiently. Apply the concepts of primitive and non-primitive data structures to select an appropriate structure for storing and managing the data and prepare a technical presentation justifying your choice based on the efficiency of data structure.
2	Choose a suitable linear or non-linear data structure for a navigation system that stores city connections and finds shortest routes between locations. Prepare a technical presentation explaining why it is appropriate.
3	A web browser supports navigating backward through previously visited pages. Recommend a suitable ADT and prepare a technical presentation explaining how it improves performance efficiency.
4	A digital library allows searching books using ISBN numbers quickly among 50,000 records. Select the most suitable data structure and Prepare a technical presentation justifying using time complexity comparison.

Code of Conduct:

I **Abinayasri K / 1921260133** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Choosing the Right Data Structure

Technical Presentation | Data Structures (CSA03)
Assessment Tool 2 | CO1

Abinayasri K | Reg. No. 1921260133
SIMATS Engineering

1 Array

2 Graph

3 Stack

4 Hash Table

All Four Problems, Four Structures

1

Student marks

1000 students

Best choice

Array

O(1) access by index

2

City navigation

Shortest route

Best choice

Graph

Dijkstra, O((V+E) log V)

3

Browser back

Previous pages

Best choice

Stack

O(1) push and pop

4

Digital library

50,000 ISBNs

Best choice

Hash Table

O(1) average search

1 Primitive vs Non-Primitive

Problem: store and manage the marks of 1000 students efficiently.

Primitive data types

int, float, char, boolean

- Hold a single value
- Fixed size, built into the language
- 1000 marks would need 1000 separate variables, which is impractical

Non-primitive structures

array, list, stack, graph

- Group many values under one name
- Built from primitive types
- Index-based access and easy loops over all marks

1000 marks of one type: store them in one non-primitive structure, an Array

1 Marks Stored in an Array



int marks[1000]; one name, 1000 slots, one slot per student

One name, 1000 slots

marks[i] holds the mark of student i, so one loop can process every record.

Direct access

Address = base + i x size, so any mark is reached in O(1) time.

Small and compact

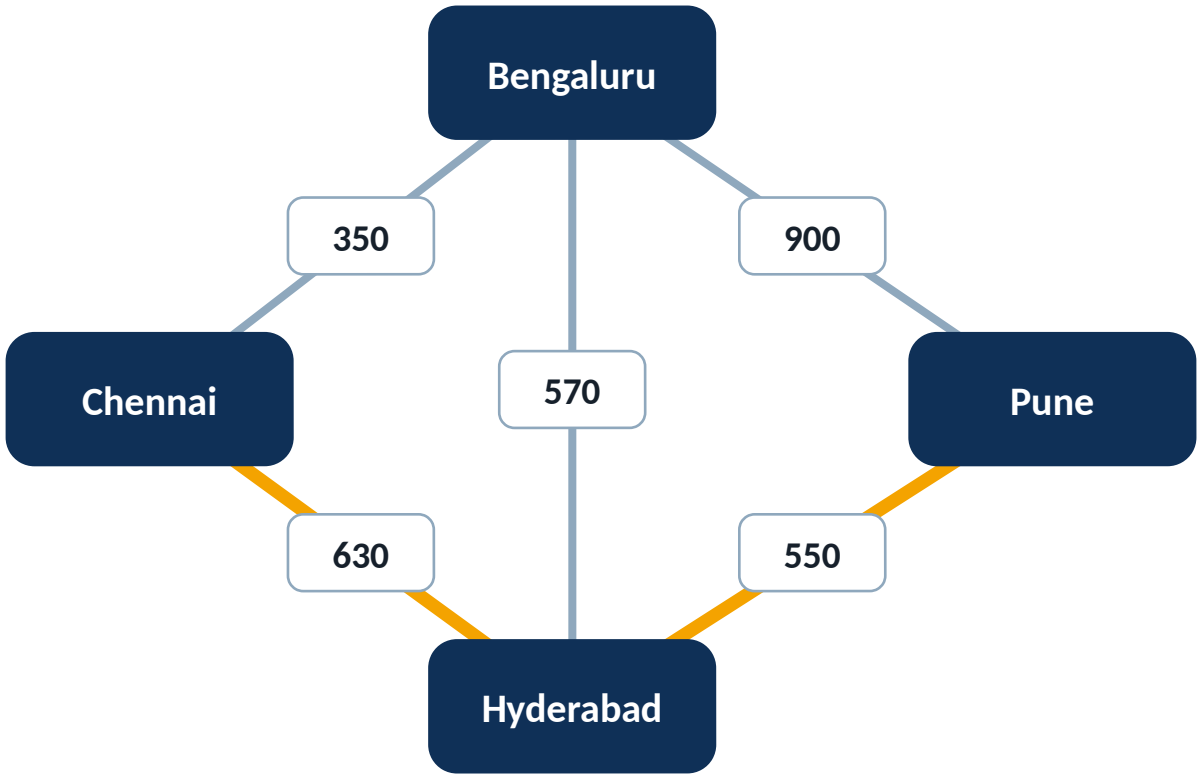
1000 x 4 bytes is about 4 KB of contiguous memory. Space is O(n).

1 Efficiency Check

Operation	Array	Linked list
Read marks of student i	**O(1)**	O(n)
Update a mark	**O(1)**	O(n)
Process all 1000 marks	**O(n)**	O(n)
Search a mark (unsorted)	**O(n)**	O(n)
Extra memory per student	**None**	One pointer
Memory layout	**Contiguous**	Scattered

The array wins on access time and memory, the two things a marks table needs most.

2 Cities as a Graph



Graph model

Vertices: cities

Edges: roads, weighted by distance

Non-linear: a city links to many cities, and routes can loop

Storage: adjacency list, O(V + E) space

Amber path: Chennai > Hyderabad > Pune = 1180 km (example distances)

2 Shortest Route with Dijkstra

1. Set the start city to 0 and every other city to infinity
2. Pick the unvisited city with the smallest distance (min-heap)
3. Relax its neighbours: keep any shorter distance found
4. Repeat until the destination city is picked

O((V + E) log V)

Time with a min-heap

O(V + E)

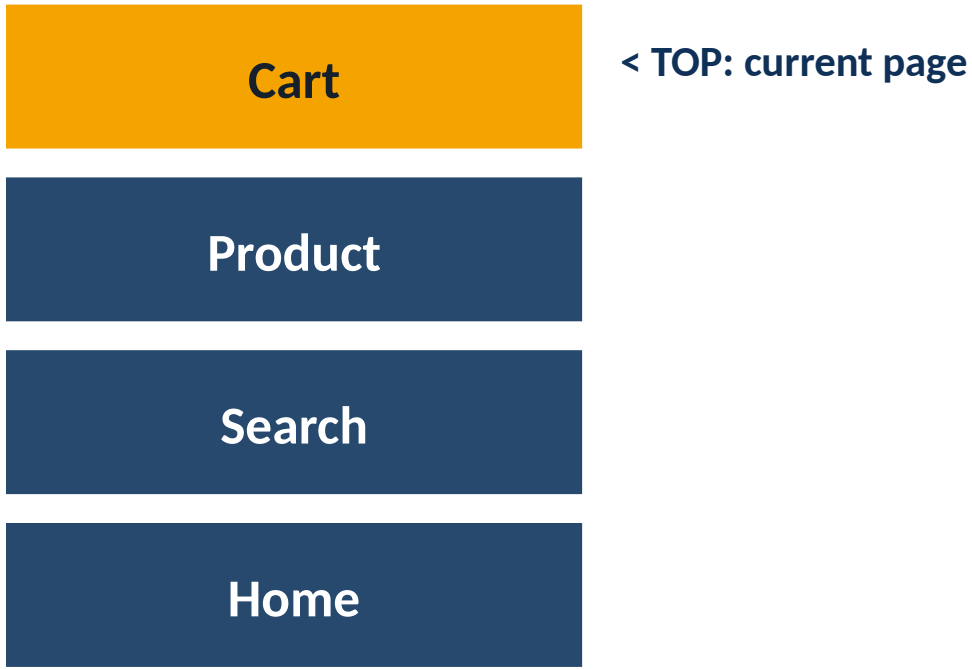
Space with an adjacency list

Why not a tree or array?

Tree: one path between two nodes, but roads form loops

Array: cannot show which cities connect

3 Browser History as a Stack



Visit a page: push(page)
Click Back: pop()

Stack ADT (LIFO)

Last in, first out: the last page visited is the first one you return to

push(page): add the new page on top

pop(): remove the top and show the page below it

peek(): read the current page

isEmpty(): no history left, so disable Back

3 Why a Stack Is Fast

O(1)
push: visit a page

O(1)
pop: go back

O(1)
peek: show current page

Why it improves efficiency

- Only the top is touched: no searching and no shifting of history
- Memory grows only with pages visited, O(n), and is freed on every pop

Why not a queue?

- A queue (FIFO) returns the oldest page, not the previous one
- For a Forward button, keep popped pages on a second stack

4 ISBN Lookup with a Hash Table

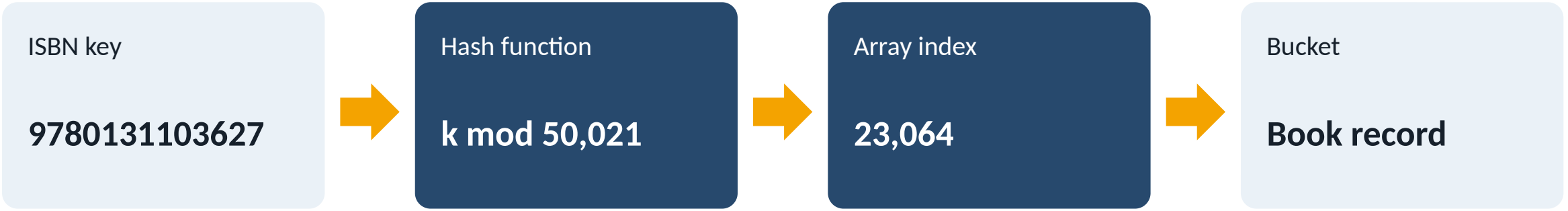


Table size

m = 50,021 (a prime), so each slot holds about one book.

Collisions

Books with the same index share a bucket as a linked list (chaining).

Operations

Insert, search and delete take O(1) time on average.

4 Time Complexity Comparison

Method	Sorted data?	Average	Worst	Steps for n = 50,000
Linear search	No	O(n)	O(n)	up to 50,000
Binary search	Yes	O(log n)	O(log n)	about 16
Binary search tree	No	O(log n)	O(n)	about 16 (balanced)
Hash table	**No**	**O(1)**	**O(n)**	**1 to 2**

Hash table worst case O(n) happens only if every key collides, which a good hash function avoids.

The hash table gives the fastest average lookup and needs no sorting.

4 Why the Hash Table Wins

50,000

comparisons, worst case

Linear search

~16

comparisons

Binary search (needs sorted data)

~1

comparison on average

Hash table

Trade-offs to remember

- Uses extra memory for the table
- Does not keep ISBNs sorted, so use a tree for range queries
- Needs a good hash function and collision handling

Summary

1. **Array**
   O(1) access by index, compact contiguous memory

2. **Graph + Dijkstra**
   Models roads and loops, shortest route in O((V + E) log V)

3. **Stack**
   Last in, first out, O(1) push and pop for Back

4. **Hash Table**
   About 1 step per ISBN lookup instead of up to 50,000

Thank you